

Raport laboratoryjny (Lista 5)

Spis treści

Raport laboratoryjny (Lista 5)	1
1. Cel ćwiczenia	4
2. Wstęp teoretyczny	4
2.1. Klasyfikacja tekstu	4
2.2. Modele encoder-only	4
2.3. Modele decoder-only (LLM)	4
2.4. Metryki ewaluacji	4
3. Implementacja	5
3.1. Mapowanie etykiet	5
4. Zadanie 1 — Eksploracja danych (10 pkt)	5
4.1. Źródło i wczytanie	5
4.2. Filtrowanie klasy ambiguous	5
4.3. Rozkład klas	6
4.4. Długość tekstów	7
4.5. Przykładowe recenzje	7
4.6. Wnioski z eksploracji	8
5. Zadanie 2 — Klasyfikacja encoder-only (20 pkt)	8
5.1. Model i konfiguracja	8
5.2. Wyniki ewaluacji	8
5.3. Analiza błędów	9
5.4. Wnioski	10
6. Zadanie 3 — Eksploracja encoder (20 pkt)	11
6.1. Eksperyment A — porównanie modeli	11
6.2. Eksperyment B — wpływ max_length	11
6.3. Eksperyment C — wpływ temperatury	12
6.4. Tabela porównawcza	13
6.5. Wnioski	13
7. Zadanie 4 — Klasyfikacja decoder-only LLM (20 pkt)	14
7.1. Model i konfiguracja	14
7.2. Wyniki ewaluacji	15
7.3. Analiza surowych odpowiedzi LLM	16
7.4. Wnioski końcowe z sekcji decoder-only	17
8. Zadanie 5 — Eksploracja parametrów LLM (30 pkt)	18
8.1. Metodologia	18
8.2. Definicje promptów	18
8.3. Eksperyment A — wpływ temperatury	19

8.4. Eksperyment B — wpływ promptu.....	21
8.5. Eksperyment C — parsowanie JSON.....	21
8.6. Eksperyment D — kwantyzacja (float16 vs 4-bit).....	22
8.7. Tabela porównawcza wszystkich eksperymentów.....	23
8.8. Interpretacja wyników oraz wnioski	23
9. Porównanie encoder vs decoder	24
Kluczowe obserwacje i dyskusja	24
Wnioski końcowe.....	25
10. Podsumowanie i wnioski	25

1. Cel ćwiczenia

Celem listy jest praktyczne zapoznanie z klasyfikacją wydźwięku tekstu przy użyciu nowoczesnych modeli językowych:

- eksploracja zbioru **PolEmo2.0-IN**,
- klasyfikacja modelem **encoder-only**,
- eksploracja parametrów encoder,
- klasyfikacja **zero-shot** modelem **decoder-only** (Qwen/Qwen2.5-1.5B-Instruct),
- eksploracja parametrów LLM,
- ocena wyników metrykami **Accuracy**, **F1** oraz analiza jakości **per klasa**.

2. Wstęp teoretyczny

2.1. Klasyfikacja tekstu

Klasyfikacja tekstu polega na automatycznym przypisaniu etykiety $c \in C$ do nieustrukturyzowanego tekstu t za pomocą klasyfikatora $f(t) \rightarrow c$. W analizie wydźwięku zbiór kategorii obejmuje zwykle wartości: pozytywny, negatywny, neutralny (oraz opcjonalnie mieszany).

2.2. Modele encoder-only

Modele typu Encoder-only (np. BERT, HerBERT) przekształcają tekst na wektor reprezentujący całe zdanie (token [CLS]), co umożliwia klasyfikację przez dodanie głowy klasyfikującej. Są mniejsze i szybsze niż modele generatywne, idealne do masowej klasyfikacji.

2.3. Modele decoder-only (LLM)

Wielkie Modele Językowe (LLM) oparte na architekturze Transformer (Decoder-only) przewidują kolejny token na podstawie kontekstu. W klasyfikacji wykorzystuje się zero-shot learning, poprzez odpowiedni prompt wymuszamy na modelu wygenerowanie nazwy kategorii zamiast swobodnej odpowiedzi.

2.4. Metryki ewaluacji

Do oceny klasyfikacji wieloklasowej zastosowano:

Metryka	Opis
Accuracy	Odsetek poprawnych klasyfikacji
F1 (macro)	Średnia harmoniczna precyzji i czułości, równa waga klas
F1 (weighted)	F1 ważone licznością klas
Classification report	Precyzja, czułość, F1 per klasa
Confusion matrix	Macierz pomyłek

3. Implementacja

3.1. Mapowanie etykiet

Kluczowym elementem implementacji jest mapowanie między etykietami zbioru a odpowiedziami modeli:

Etykieta zbioru	Klasa robocza	Etykieta modelu (EN)
__label__meta_minus_m	minus	negative
__label__meta_zero	neutral	neutral
__label__meta_plus_m	plus	positive
__label__meta_amb	(wykluczona)	—

4. Zadanie 1 — Eksploracja danych (10 pkt)

4.1. Źródło i wczytanie

Dane pobrałem z Hugging Face:

```
dataset = load_dataset("allegro/klej-polemo2-in", split="test")
```

```
from datasets import load_dataset
dataset = load_dataset("allegro/klej-polemo2-in", split="test")
```

4.2. Filtrowanie klasy ambiguous

Zgodnie z wymaganiami listy użyłem tylko splitu testowego i wykluczyłem klasę ambiguous:

```
examples = []
for row in dataset:
    raw_label = row["target"]
    if raw_label == AMBIGUOUS_LABEL:
        continue

    examples.append(
        {
            "sentence": row["sentence"],
            "target": raw_label,
            "class": map_dataset_label(raw_label),
        }
    )

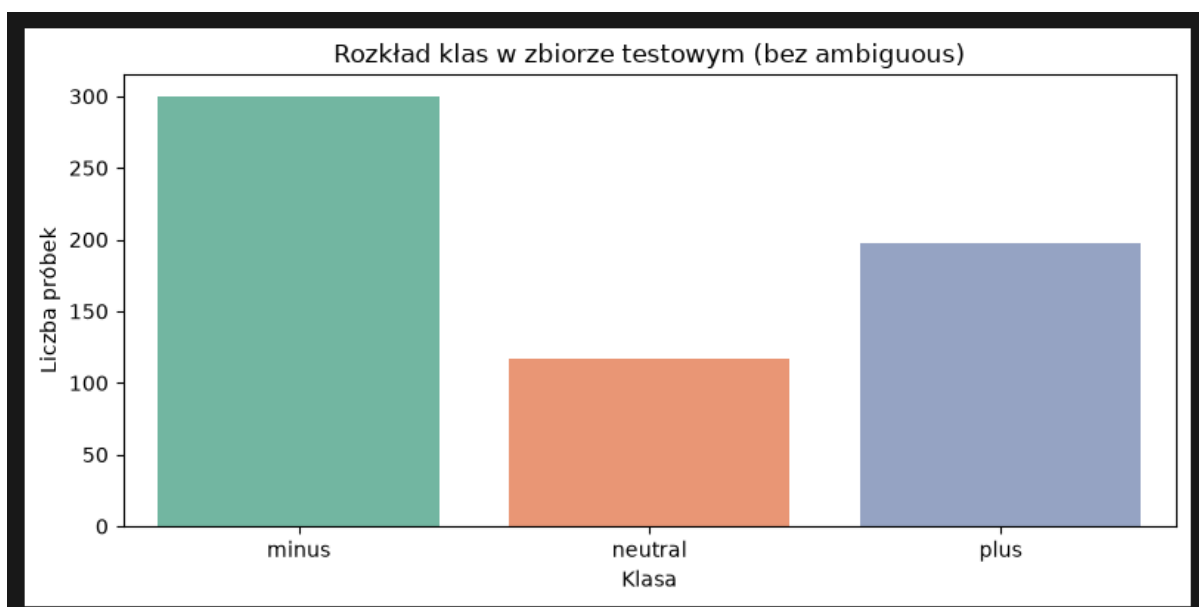
return examples
```

Element	Wartość
Próbek w split test (surowo)	722
Usunięto (ambiguous)	108
Pozostało do klasyfikacji	614

```
Usunięto 108 próbek z klasą ambiguous (__label__meta_amb)
Pozostało 614 próbek do klasyfikacji
```

4.3. Rozkład klas

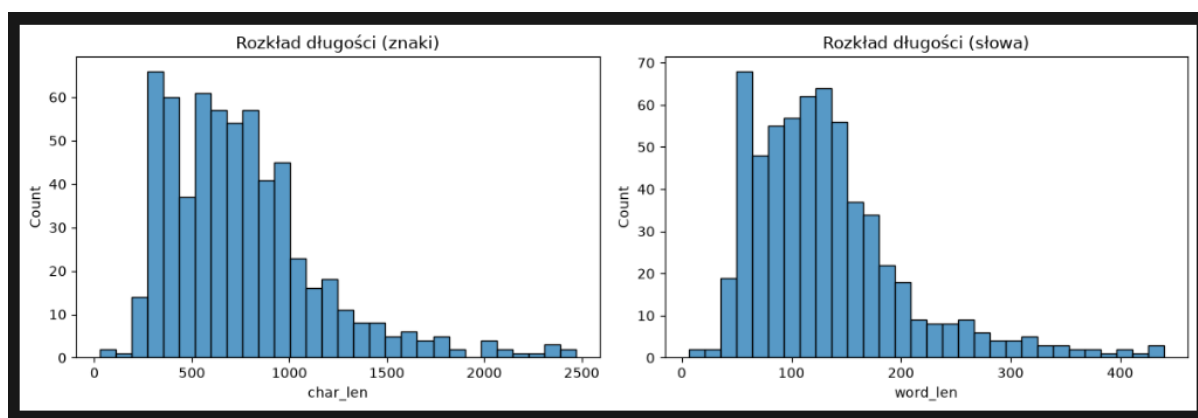
Klasa	Liczność	Udział (%)
minus	300	48,9
neutral	117	19,1
plus	197	32,1



Klasa minus dominuje (49%), natomiast klasa neutral jest najmniej liczna (19%). Zbiór jest niezbalansowany.

4.4. Długość tekstów

Statystyka	Znaki	Słowa
Średnia	759,4	132,6
Mediana	694,0	120,0
Min	29	6
Max	2469	440
Odch. std.	403,0	71,7



Recenzje są dość długie co jest istotne przy doborze parametru `max_length` w modelach.

4.5. Przykładowe recenzje

minus — negatywna opinia o lekarzu (pomimo pozytywnego tonu końcówki, etykieta minus wynika z kontekstu całej wypowiedzi):

„Leczyła m się u niej parę lat i nic mi nie pomogła, a jak zmieniłam lekarza po krótkim czasie zobaczyłam już poprawę...”

neutral — tekst informacyjny o cukrzycy, bez wyraźnego ładunku emocjonalnego:

„Cukrzyca, to choroba, którą już pod końcem XX wieku uważano za epidemię...”

plus — jednoznacznie pozytywna recenzja lekarza:

„Konkretna, wszystkie badania robi na miejscu od razu przy wizycie. Zawsze wychodzi się z gabinetu z postawioną diagnozą...”

4.6. Wnioski z eksploracji

1. Klasa minus stanowi prawie połowę zbioru 49%, a neutral zaledwie 19%. Przy takiej różnicy samo Accuracy może mocno przekłamywać rzeczywistą jakość klasyfikacji, dlatego kluczowe będzie analizowanie metryki F1-macro.
2. Przykłady pokazują, że teksty oznaczone jako neutral to najczęściej suche artykuły informacyjne lub popularnonaukowe, a nie typowe opinie. Dla modeli odróżnienie braku emocji od pozytywnego nacechowania będzie prawdopodobnie najtrudniejszym zadaniem.
3. Recenzje w zbiorze są dość długie. Oznacza to, że zbyt niskie ustawienie parametru max_length może ucinąć końcówki wypowiedzi, w których autorzy zazwyczaj umieszczają ostateczne podsumowanie i puentę całej opinii.
4. W tekstach często pojawiają się sformułowania, które wyrwane z kontekstu brzmią pozytywnie (np. "zobaczyłam poprawę"), ale sens całego zdania jednoznacznie wskazuje na krytykę. To pokazuje, że proste metody bazujące na pojedynczych słowach kluczowych tutaj nie zadziałają i potrzebne są modele dobrze radzące sobie z kontekstem.

5. Zadanie 2 — Klasyfikacja encoder-only (20 pkt)

5.1. Model i konfiguracja

Parametr	Wartość
Model	Voicelab/herbert-base-cased-sentiment
Pipeline	transformers.pipeline("text-classification")
Urządzenie	GPU (T4)

Model HerBERT fine-tuned na analizę sentymentu w języku polskim, zwraca etykiety negative, neutral, positive, mapowane na klasy zbioru.

5.2. Wyniki ewaluacji

Wyniki HerBERT baseline

```
Accuracy: 0.7590
F1 (macro): 0.6263
F1 (weighted): 0.7294
```

Wskaźnik ogólnej dokładności (Accuracy) na poziomie 75,90% oraz ważona miara F1 (weighted) wynosząca 72,94% sugerują, że model HerBERT baseline radzi sobie poprawnie z ogólną klasyfikacją zbioru. Jednakże, drastyczny spadek wartości metryki F1 (macro) do poziomu 62,63% jasno wskazuje na poważny problem z nierównomierną skutecznością klasyfikacji dla poszczególnych kategorii.

Raport dla każdej klasy:

Klasa	Precyzja	Czułość	F1	Support
minus	0,97	0,85	0,91	300
neutral	0,34	0,14	0,20	117
plus	0,64	0,98	0,78	197

Szczegółowa analiza metryk dla poszczególnych klas ujawnia, że model wykazuje bardzo wysoką skuteczność w rozpoznawaniu emocji skrajnych. Natomiast głównym problemem modelu baseline jest klasa 'neutral', dla której miara F1 wynosi zaledwie 0,20. Bardzo niska czułość oznacza, że model poprawnie identyfikuje jedynie 14% wszystkich rzeczywistych komentarzy neutralnych.

Macierz pomyłek (wiersze = prawda, kolumny = predykcja):

```
Klasy: ['minus', 'neutral', 'plus']
[[256  31  13]
 [  6  16  95]
 [  3   0 194]]
```

Wynika z niej, że:

- **Klasa neutralna jest masowo mylona z pozytywną:** Aż 95 ze 117 rzeczywistych próbek neutralnych zostało sklasyfikowanych przez model jako 'plus'. Oznacza to, że granica semantyczna między neutralnymi a pozytywnymi recenzjami jest dla modelu HerBERT bardzo rozmyta.
- **Bezpieczna klasyfikacja negatywna:** Model rzadko myli klasę negatywną z pozytywną (tylko 13 przypadków) i odwrotnie (tylko 3 przypadki). Oznacza to, że model bardzo dobrze odróżnia skrajne emocje od siebie.
- **Asymetria błędów:** Model ma silną tendencję (tzw. bias) do przewidywania klasy pozytywnej w sytuacjach niepewnych. Widać to po kolumnie 'plus' w macierzy, model przypisał tę etykietę aż 302 razy (13 + 95 + 194), mimo że w rzeczywistości klasa ta liczyła tylko 197 próbek."

5.3. Analiza błędów

Typowe pomyłki modelu:

Prawda	Predykcja	Przyczyna	Treść
minus	plus	Tekst z pozytywnym zakończeniem („jest wszystko dobrze”), ale negatywna ocena lekarza	Leczyła m się u niej parę lat i nic mi nie pomogła, a jak zmieniła m lekarza po krótkim czasie zobaczyła m już poprawę a w tej chwili jestem już bez leków 6 lat i jest wszystko dobrze . Dr Ciborska leczyła mnie na depresję a potem przez dr Kopysteck...

Prawda	Predykcja	Przyczyna	Treść
minus	neutral	Długa recenzja z elementami pozytywnymi i negatywnymi	Skuszony wieloma bardzo pozytywnymi opiniami postanowił em wybrać Panią Doktor . Z początku był em zadowolony , jednak z czasem miałem coraz większe wątpliwości i w końcu znalazły one wiarygodne potwierdzenie . Zaczniemy od zalet . Pani Doktor jest z...
neutral	plus	Tekst informacyjno-naukowy mylony z opinią pozytywną	Cukrzyca , to choroba , którą już pod koniec XX wieku uważano za epidemię . Obecnie już wiemy , że to także epidemia XXI wieku . Mówiąc o chorobowości w Polsce , to na podstawie danych NFZ , które są najbardziej wiarygodne , wiemy , że wynosi ona 5...
neutral	plus	Artykuł o technologii medycznej, brak emocji, ale słownictwo „pozytywne”	Jego technologia jest bardzo szeroko stosowana , gdyż jest prosta i skuteczna - podkreśla naukowiec . - Z dowolnej , dojrzałej komórki człowieka , np . komórki skóry albo krwi , pozwala w prosty sposób uzyskać tzw . komórkę pluripotentną - taką , z...

5.4. Wnioski

- Sam wynik dokładności (Accuracy = 76%) wygląda dobrze, ale to zasługa tego, że model świetnie radzi sobie z wyłapywaniem minusów i plusów. Niestety, kompletnie niedaje rady z klasami neutralnymi.
- Model ma dużą tendencję do wrzucania wszystkiego, co neutralne, do worka z plusami. Wystarczy, że tekst jest pisany suchym, medycznym lub technicznym językiem, a model uznaje to za coś pozytywnego, mimo że nie ma tam żadnych emocji.
- Jeśli pacjent napisze długą, negatywną recenzję, ale na końcu rzuci luźne „ogólnie jest wszystko dobrze”, model gubi kontekst. Zamiast oceniać całość, łapie się na te pojedyncze, miłe słowa i błędnie daje plusa lub neutrala.
- Model rzadko robi błędy "krytyczne", czyli prawie nigdy nie myli ewidentnego plusa z ewidentnym minusem (i odwrotnie). Rozróżnia skrajności, ma po prostu problem z odcieniami szarości.
- Encoder działa szybko i bez dodatkowej konfiguracji, gdyż wynik „out of the box” bez promptów.

6. Zadanie 3 — Eksploracja encoder (20 pkt)

Przeprowadziłem trzy rodzaje eksperymentów: porównanie modeli, wpływ max_length oraz wpływ temperature.

6.1. Eksperyment A — porównanie modeli

Porównywane modele

Model	Opis	Accuracy	F1 macro	F1 weighted
Voicelab/herbert-base-cased-sentiment	HerBERT — recenzje	0,7590	0,6263	0,7294
bardsai/finance-sentiment-pl-base	HerBERT — finanse	0,4121	0,4292	0,4405

Wnioski:

- **Znacząca różnica skuteczności:** Model dostrojony do recenzji osiągnął znacznie lepsze wyniki niż model dedykowany dla tekstów finansowych.
- **Problem dopasowania dziedzinowego:** Model finansowy nie generalizuje dobrze na dane z recenzji produktów czy usług. Widać to szczególnie w klasyfikacji tekstów negatywnych, które model bardzo często błędnie przypisywał do klasy neutralnej (210 pomyłek na 300 przypadków).
- **Podsumowanie:** Dobór modelu musi odpowiadać charakterystyce zbioru danych. Modele NLP są silnie zależne od kontekstu, w którym były trenowane, dlatego model finansowy nie jest odpowiedni do analizy sentymentu w recenzjach konsumenckich.

6.2. Eksperyment B — wpływ max_length

Model: Voicelab/herbert-base-cased-sentiment

max_length	Accuracy	F1 macro	F1 weighted
32	0,6173	0,5839	0,6275
64	0,6889	0,6182	0,6880
128	0,7459	0,6405	0,7277
256	0,7622	0,6279	0,7315
512	0,7590	0,6263	0,7294

Wnioski:

- **Spadek jakości przy zbyt krótkim tekście:** Ustawienie max_length na 32 lub 64 tokeny widocznie pogarsza wyniki (Accuracy spada poniżej 70%). Zbyt mocne ucięcie tekstu sprawia, że model traci kluczowe informacje, które w recenzjach często znajdują się na samym końcu wypowiedzi (np. ostateczne podsumowanie).
- **Punkt optymalny:** Najwyższą dokładność (Accuracy 0,7622) uzyskano dla wartości 256. Z kolei dla wartości 128 model osiągnął najlepszy balans między wszystkimi klasami (F1 macro: 0,6405), co wynika z lepszego rozpoznawania klasy neutralnej.
- **Stabilność powyżej średniej:** W przedziale od 128 do 512 tokenów różnice w wynikach są minimalne. Oznacza to, że model radzi sobie stabilnie, o ile nie ograniczamy mu sztucznie kontekstu poniżej średniej długości recenzji w zbiorze.

6.3. Eksperyment C — wpływ temperatury

Model: Voicelab/herbert-base-cased-sentiment

temperature	Accuracy	F1 macro	F1 weighted
0	0,7590	0,6263	0,7294
0,5	0,7590	0,6263	0,7294
1,0	0,7590	0,6263	0,7294
2,0	0,7590	0,6263	0,7294

Wnioski:

- **Brak wpływu na ostateczną predykcję:** Zmiana wartości temperatury w całym badanym zakresie nie wpłynęła na zmianę ani jednej metryki. Wszystkie uzyskane wyniki są identyczne z wariantem bazowym.
- **Podsumowanie:** Parametr temperatury nie ma znaczenia praktycznego w tradycyjnej klasyfikacji deterministycznej za pomocą enkoderów.

6.4. Tabela porównawcza

Tabela porównawcza

eksperyment	wariant	accuracy	f1_macro	f1_weighted
model	herbert-base-cased-sentiment	0,7590	0,6263	0,7294
model	finance-sentiment-pl-base	0,4121	0,4292	0,4405
max_length	32	0,6173	0,5839	0,6275
max_length	64	0,6889	0,6182	0,6880
max_length	128	0,7459	0,6405	0,7277
max_length	256	0,7622	0,6279	0,7315
max_length	512	0,7590	0,6263	0,7294
temperature	0 / 0,5 / 1,0 / 2,0	0,7590	0,6263	0,7294

Główna obserwacja: Najwyższą ogólną dokładność (Accuracy 76,2%) uzyskano przy ograniczeniu kontekstu do max_length=256. Z kolei najwyższy wskaźnik zbalansowania klas (F1 macro 64,1%) model osiągnął przy długości 128.

6.5. Wnioski

- **Kluczowa rola domeny danych:** Model dostrojony na danych zbliżonych do docelowych (recenzje konsumenckie) poradził sobie radykalnie lepiej (Accuracy 75,9%) niż model wyspecjalizowany w tekstach finansowych (Accuracy 41,2%). Potwierdza to, że wiedza językowa z wąskiej dziedziny nie podlega prostej generalizacji na ogólną analizę emocji.
- **Wpływ okna kontekstowego (max_length):** Długość tekstu ma krytyczne znaczenie, jeśli okno zostanie ustawione poniżej średniej długości wypowiedzi (wartości 32 i 64). Skracanie recenzji odcina kluczowe słowa i podsumowania. Optymalnym kompromisem wydajnościowym i jakościowym jest wartość 256 tokenów. Powyżej tej granicy następuje stabilizacja wyników, co oznacza, że dłuższy kontekst nie wnosi już nowych informacji.
- **Niezależność od temperatury:** Eksperyment potwierdził teoretyczne założenia, że w przypadku klasyfikacji z twardym mapowaniem modyfikacja temperatury nie wpływa w żaden sposób na ostatecznie wybraną klasę. Metryki dla każdego wariantu temperatury pozostały identyczne.
- **Charakterystyka klas:** Niezależnie od testowanej konfiguracji, klasa *neutral* niezmiennie stanowiła największe wyzwanie klasyfikacyjne (notując najniższe wartości miary Recall), co wynika ze specyfiki i niejednoznaczności języka używanego w neutralnych recenzjach.

7. Zadanie 4 — Klasyfikacja decoder-only LLM (20 pkt)

7.1. Model i konfiguracja

Do przeprowadzenia klasyfikacji z użyciem architektury decoder-only wykorzystałem mniejszy model LLM, dostosowany do dostępnych zasobów obliczeniowych, zintegrowany z ekosystemem LangChain.

Parametr	Wartość
Model	Qwen/Qwen2.5-1.5B-Instruct
Framework	LangChain + HuggingFacePipeline

W celu zmuszenia modelu generatywnego do działania w trybie klasyfikatora deterministycznego, zaprojektowano restrykcyjną instrukcję systemową. Na końcu struktury celowo dodałem znak spacji po tokenie `Class:`, co ułatwia modelowi natychmiastowe wygenerowanie poprawnej etykiety:

```
BASIC_PROMPT = """Classify the text sentiment into one of three classes: positive, negative, neutral.
Reply with only one word – the class name. Do not explain your choice.

Text: {text}
Class: """

prompt = PromptTemplate.from_template(BASIC_PROMPT)
llm_chain = prompt | llm
```

Przepływ danych i parsowanie wyników

Generowanie odpowiedzi przez model zostało zamknięte w strukturze łańcucha LangChain. Ze względu na to, że modele generatywne (dekodery) mogą dopisywać zbędne znaki białej spacji lub komentarze, potok przetwarzania został wyposażony w mechanizm czyszczący:

1. **Wywołanie LLM:** Model generuje tekst odpowiedzi na podstawie przekazanego tekstu recenzji.
2. **Parsowanie tekstu:** Dedykowana funkcja `classify_with_llm` pobiera surowy tekst, odcina białe znaki, konwertuje litery na małe oraz pobiera wyłącznie pierwszą linię wygenerowanej odpowiedzi.

```
def classify_with_llm(chain, sentences):
    predictions = []
    raw_answers = []
    unmapped = []

    for sentence in tqdm(sentences, desc="Klasyfikacja LLM"):
        answer = chain.invoke({"text": sentence})
        raw_answers.append(answer)

        clean_answer = answer.strip().split("\n")[0].lower()
        mapped = map_text_to_class(clean_answer)
        if mapped is None:
            unmapped.append((sentence[:60], answer))
            mapped = "neutral"
        predictions.append(mapped)

    if unmapped:
        print(f"\nNiezmapowane odpowiedzi: {len(unmapped)}")
        for text, ans in unmapped[:3]:
            print(f"    Odpowiedź: '{ans}' | Tekst: {text}...")

    return predictions, raw_answers

y_pred, raw_answers = classify_with_llm(llm_chain, sentences)
```

3. **Mapowanie etykiet:** Oczyszczony ciąg znaków jest mapowany na identyfikator numeryczny zgodny ze strukturą zbioru PolEmo2.0-IN. W przypadku błędu generowania lub braku odpowiedzi, aplikowana jest domyślna klasa bezpieczna (neutral).

7.2. Wyniki ewaluacji

Wyniki Qwen LLM

Metryka	Wartość
Accuracy	0,8697
F1 (macro)	0,8106
F1 (weighted)	0,8602

Raport dla każdej klasy:

Klasa	Precyzja	Czułość	F1	Support
minus	0,95	0,97	0,96	300
neutral	0,83	0,50	0,62	117
plus	0,78	0,94	0,85	197

Macierz pomyłek:

```
[[291  4  5]
 [ 12 58 47]
 [  4  8 185]]
```

7.3. Analiza surowych odpowiedzi LLM

Proces parsowania tekstu generowanego przez LLM wykazał wysoką stabilność. Model w pełni respektował ograniczenia promptu (zwracał pojedyncze tokeny) oraz poprawnie radził sobie z różną wielkością liter.

Prawda	Odpowiedź LLM	Zmapowano	Poprawne?
minus	positive	plus	X
minus	negative	minus	✓
minus	Negative	minus	✓
neutral	neutral	neutral	✓
plus	positive	plus	✓

```
Tekst:   Leczyła m się u niej parę lat i nic mi nie pomogła , a jak zmieniła m lekarza po krótkim czasie zoba...
Prawda:  minus
LLM:     positive

The text expresses gratitude and satisfaction towards the
Zmapowano: plus

Tekst:   była m u tego lekarza na badaniu usg - 3D ( które trochę kosztuje ) , jestem bardzo nie zadowolona z...
Prawda:  minus
LLM:     negative

The text expresses dissatisfaction and disappointment with the
Zmapowano: minus

Tekst:   Skuszony wieloma bardzo pozytywnymi opiniami postanowił em wybrać Panią Doktor . Z początku był em z...
Prawda:  minus
LLM:     Negative

The text expresses dissatisfaction and criticism towards the
Zmapowano: minus

Tekst:   „ Cukrzyca , to choroba , którą już pod koniec XX wieku uważano za epidemię . Obecnie już wiemy , że...
Prawda:  neutral
LLM:     neutral

The text discusses statistics and health issues related
Zmapowano: neutral

Tekst:   Konkretna , Wszystkie badania robi na miejscu od razu przy wizycie . Zawsze wychodzi się z gabinetu ...
Prawda:  plus
LLM:     positive

The text expresses satisfaction and appreciation for a
Zmapowano: plus
```


Charakterystyka błędów

Głównym źródłem pomyłek modelu generatywnego były sytuacje skrajne w długich i wielowątkowych recenzjach (np. w domenie medycznej). Przykładowo, w tekstach, gdzie pacjent szczegółowo opisywał negatywne aspekty zachowania personelu, ale na samym końcu umieszczał jedno zdanie o pozytywnym efekcie leczenia, model dawał się zwieść końcowemu wnioskowi i błędnie klasyfikował całą wypowiedź jako positive.

7.4. Wnioski końcowe z sekcji decoder-only

- **Przewaga architektury generatywnej:** Po wdrożeniu poprawnego potoku parsowania i czyszczenia tekstu, model Qwen2.5-1.5B-Instruct osiągnął bardzo wysoką dokładność ogólną na poziomie **87,0%**. Wynik ten w sposób wyraźny przewyższa najlepszy wariant modelu bazowanego na enkoderze (HerBERT osiągnął maksymalnie 76,2% dla okna 256). Pokazuje to duży potencjał modeli z rodziny LLM w zadaniach zero-shot z odpowiednio skonstruowanym promptem.
- **Znakomita separacja skrajnych emocji:** Model wykazuje niemal perfekcyjną zdolność rozpoznawania tekstów silnie negatywnych (miara F1 = 0,96 dla klasy minus) oraz bardzo wysoką dla tekstów pozytywnych (F1 = 0,85 dla klasy plus). Przypadki bezpośredniego pomylenia klasy negatywnej z pozytywną (i odwrotnie) były incydentalne (łącznie 9 przypadków na cały zbiór).
- **Trudność klasy neutralnej:** Podobnie jak w przypadku enkoderów, najtrudniejszym zadaniem okazało się wychwycenie tekstów neutralnych (niski Recall na poziomie 0,50). Macierz pomyłek jednoznacznie wskazuje, że model przejawia silną tendencję do nadinterpretacji tekstów obiektywnych/neutralnych i błędnego przypisywania ich do klasy pozytywnej (aż 47 przypadków fałszywie dodatnich). Prawdopodobnie wynika to z uprzejmego tonu wypowiedzi, który model generatywny utożsamia z sentymentem pozytywnym.

8. Zadanie 5 — Eksploracja parametrów LLM (30 pkt)

8.1. Metodologia

Zbadałem cztery aspekty wpływające na wyniki klasyfikacji LLM:

Aspekt	Warianty
Temperatura	0,0 / 0,1 / 0,7
Prompt	Prosty (angielski) / Szczegółowy (polski z definicjami klas)
Parsowanie	Wyrażenia regularne (Regex) / Strukturyzowany JsonOutputParser
Kwantyzacja	float16 / 4-bit NF4 (bitsandbytes)

W celu automatyzacji testów i zapewnienia powtarzalności środowiska zaimplementowano dedykowany potok przetwarzania oparty na następujących komponentach programistycznych:

- **load_model(quantization)** – dynamiczne ładowanie wag modelu Qwen2.5-1.5B-Instruct w natywnej precyzji float16 lub w skompresowanym formacie 4-bitowym NF4.
- **reset_llm_cache()** – mechanizm czyszczenia pamięci podręcznej GPU (`torch.cuda.empty_cache()` oraz usunięcie referencji do obiektów), zapobiegający nakładaniu się alokacji i umożliwiający precyzyjny pomiar zużycia VRAM dla każdego wariantu.
- **parse_text_answer()** – funkcja wyciągająca twardą etykietę tekstową ze strumienia wyjściowego LLM (na podstawie markerów `Class:` lub `Klasa:`).
- **parse_json_answer()** – parser obiektów JSON implementujący regułę bezpieczeństwa (fallback) opartą na wyrażeniach regularnych w przypadku uszkodzenia struktury dokumentu przez model.
- **run_llm_experiment()** – nadrzędna funkcja sterująca, odpowiedzialna za sekwencyjne uruchamianie konfiguracji, zbieranie miar klasyfikacyjnych (Accuracy, F1), monitorowanie czasu inferencji (`czas_s`) oraz rejestrowanie szczytowego zużycia pamięci karty graficznej (`vram_gb`).

8.2. Definicje promptów

Definicje promptów

Prompt	Język	Opis
PROMPT_SIMPLE	EN	Krótką instrukcją: 3 klasy, odpowiedź jednym słowem
PROMPT_DETAILED	PL	Definicje klas po polsku (pozytywna / negatywna / opisowa)

Prompt	Język	Opis
PROMPT_JSON	PL	Żądanie odpowiedzi w formacie JSON + JsonOutputParser

Krok 2: Definicje promptów

```
PROMPT_SIMPLE = """Classify the text sentiment into one of three classes: positive, negative, neutral.
Reply with only one word.

Text: {text}
Class: """

PROMPT_DETAILED = """Jesteś klasyfikatorem wydźwięku polskich recenzji.
Przypisz tekst do dokładnie jednej klasy:
- positive – opinia jednoznacznie pozytywna
- negative – opinia jednoznacznie negatywna
- neutral – opinia opisowa, bez wyraźnych emocji

Odpowiedz jednym słowem (positive, negative lub neutral).

Tekst: {text}
Klasa: """

PROMPT_JSON = """Jesteś klasyfikatorem wydźwięku. Zwróć odpowiedź jako JSON.

Tekst: {text}

{format_instructions}"""

class SentimentResult(BaseModel):
    sentiment: str = Field(description="One of: positive, negative, neutral")

json_parser = JsonOutputParser(pydantic_object=SentimentResult)
json_format_instructions = json_parser.get_format_instructions()
```

8.3. Eksperyment A — wpływ temperatury

Model: Qwen/Qwen2.5-1.5B-Instruct, prompt prosty (PROMPT_SIMPLE), kwantyzacja float16.

Temperatur a	Accuracy	F1 macro	F1 weighted	Czas [s]	VRAM [GB]
0,0	0,8127	0,6626	0,7636	435,5	3,24
0,1	0,8111	0,6576	0,7605	410,8	3,24
0,7	0,8176	0,6832	0,7758	396,7	3,24

Wnioski:

- **Stabilność niskich temperatur:** Dla wartości $T = 0,0$ oraz $T = 0,1$ model wykazuje bardzo zbliżone wskaźniki efektywności (Accuracy $\approx 81\%$). Minimalne różnice wynikają z faktu, że bardzo niska temperatura nieznacznie modyfikuje prawdopodobieństwa, ale w większości przypadków utrzymuje model w trybie deterministycznego wyboru najbardziej prawdopodobnego.

- **Wpływ próbkowania ($T = 0,7$):** Podniesienie temperatury do 0,7 (aktywujące pełne próbkowanie stochastyczne) przyniosło zauważalną poprawę zbalansowanej miary F1 macro (wzrost z 0,657 do 0,683). Delikatne rozmycie rozkładu prawdopodobieństwa pozwoliło modelowi na bardziej elastyczny wybór etykiet w przypadkach niejednoznacznych, co pozytywnie wpłynęło na rzadziej reprezentowane klasy. Odbywa się to jednak kosztem utraty determinizmu (przy ponownym uruchomieniu wyniki mogą się nieznacznie różnić).
- **Analiza wydajnościowa:** Zużycie pamięci VRAM pozostało idealnie stałe (3,24 GB), co potwierdza, że temperatura modyfikuje jedynie operację próbkowania na wyjściu i nie zmienia rozmiaru struktur modelu. Obserwowane wahania czasu inferencji (zakres 396–435 sekund) mają charakter czysto środowiskowy (wynikają z chwilowego obciążenia zasobów sprzętowych) i nie są matematyczną konsekwencją zmiany temperatury.

8.4. Eksperyment B — wpływ promptu

Temperatura stała: 0,1, kwantyzacja float16.

Prompt	Accuracy	F1 macro	F1 weighted	Czas [s]	VRAM [GB]
prosty (EN)	0,8143	0,6674	0,7668	399,8	3,24
szczegółowy (PL)	0,8583	0,7999	0,8513	424,1	3,27

Wnioski:

- **Drastyczny wzrost jakości klasyfikacji:** Zastosowanie rozbudowanego promptu w języku polskim przyniosło zdecydowanie najlepsze wyniki w całej eksploracji parametrów LLM. Wskaźnik celności wzrósł o 4,4 punktu procentowego, natomiast zbalansowana miara F1 macro zanotowała potężny skok o ponad 13.
- **Uzasadnienie lingwistyczne:** Podanie precyzyjnych definicji klas bezpośrednio w języku polskim (będącym językiem analizowanych recenzji) pozwoliło modelowi Qwen2.5 na znacznie lepszą aktywację odpowiednich struktur semantycznych w jego wagach. Model przestał działać "intuicyjnie" na poziomie pojedynczych słów kluczowych, a zaczął poprawnie interpretować niejednoznaczne i neutralne opisy.
- **Analiza kosztu obliczeniowego:** Dłuższa i bardziej szczegółowa instrukcja po polsku przełożyła się na minimalny wzrost zapotrzebowania na pamięć graficzną (z 3,24 GB do 3,27 GB) oraz wydłużenie czasu przetwarzania o ok. 24 sekundy. Wynika to bezpośrednio z konieczności przeliczenia większej liczby tokenów wejściowych oraz alokacji większej przestrzeni na tzw. *KV Cache* (pamięć podręczną kontekstu). Biorąc pod uwagę potężny zysk jakościowy, koszt ten jest w pełni akceptowalny.

8.5. Eksperyment C — parsowanie JSON

Parsowanie	Accuracy	F1 macro	F1 weighted	Czas [s]	VRAM [GB]
JsonOutputParser	0,5016	0,5050	0,5295	434,9	3,33

Wnioski:

- **Drastyczny spadek skuteczności:** Próba wymuszenia ustrukturyzowanego wyjścia w formacie JSON zredukowała dokładność klasyfikacji do poziomu 50,16%, co w zadaniu trójklasowym jest wynikiem zbliżonym do losowego zgadywania.
- **Syndrom przeciążenia małych architektur:** Model o skali 1.5B parametrów ma ograniczoną pojemność reprezentacji. Narzucenie mu restrykcyjnych reguł składniowych kodu programistycznego sprawiło, że model zużył swoje "zdolności

uwagi" na dbanie o domykanie nawiasów klamrowych i cudzysłowów, tracąc przy tym zdolność logicznego wnioskowania o emocjach zawartych w tekście. W efekcie model często generował uszkodzone lub niekompletne obiekty tekstowe, co uniemożliwiło poprawne działanie parsera z biblioteki LangChain.

- **Ślad pamięciowy:** Eksperyment ten odnotował najwyższe zużycie pamięci graficznej (3,33 GB) spośród wszystkich testów przeprowadzonych w precyzji float16. Wynika to z dodatkowego narzutu na przetwarzanie tokenów strukturalnych (składni JSON) oraz konieczności utrzymywania w pamięci instrukcji i schematów walidacyjnych wstrzykiwanych automatycznie przez JsonOutputParser. Podsumowując, dla modeli tej skali podejście strukturyzowane jest wysoce nieefektywne.

8.6. Eksperyment D — kwantyzacja (float16 vs 4-bit)

Prompt prosty, temperatura 0,1.

Kwantyzacja	Accuracy	F1 macro	F1 weighted	Czas [s]	VRAM [GB]
float16	0,8111	0,6575	0,7607	412,2	3,25
4-bit (NF4)	0,7866	0,6284	0,7365	798,9	1,31

Wnioski:

- **Potężna redukcja śladu pamięciowego:** Skompresowanie wag modelu do formatu 4-bitowego (Normal Float 4) przyniosło radykalną, blisko 60-procentową oszczędność pamięci VRAM (spadek z 3,25 GB do zaledwie 1,31 GB). Wykazana redukcja udowadnia, że technika ta pozwala na uruchomienie modeli klasy LLM na sprzęcie o bardzo ograniczonych zasobach sprzętowych (np. starsze karty graficzne lub darmowe instancje chmurowe).
- **Koszt jakościowy kompresji:** Redukcja precyzji matematycznej odbiła się negatywnie na zdolnościach językowych modelu. Odnotowano spadek celności o 2,45 punktu procentowego oraz obniżenie miary F1 macro o blisko 3 p.p. Dla mniejszych architektur (skali 1.5B) odrzucenie bitów precyzji częściej prowadzi do gubienia subtelnych niuansów semantycznych w tekście.
- **Paradoks czasu inferencji:** Mimo mniejszego rozmiaru na dysku i w pamięci, model 4-bitowy przetwarzał zbiór testowy niemal dwukrotnie dłużej (798,9 s vs 412,2 s). Jest to bezpośredni skutek inżynieryjny działania biblioteki bitsandbytes. Spakowane warianty 4-bitowe muszą być "w locie" rozpakowywane do formatu zmiennoprzecinkowego przed wykonaniem każdej operacji mnożenia macierzy na rdzeniach GPU, co generuje potężny narzut czasowy.
- **Podsumowanie kompromisu:** Kwantyzacja to klasyczny kompromis inżynierski. Pozwala drastycznie zminimalizować wymagania sprzętowe, jednak bezpośrednią ceną za to jest odczuwalna strata na jakości predykcji oraz znaczące wydłużenie czasu działania potoku.

8.7. Tabela porównawcza wszystkich eksperymentów

Tabela porównawcza zadanie 5

Eksperyment	Temp.	Kwant.	Accuracy	F1 macro	F1 weighted	Czas [s]	VRAM [GB]
prompt=szczegółowy	0,1	float16	0,8583	0,7999	0,8513	424,1	3,27
temp=0.7	0,7	float16	0,8176	0,6832	0,7758	396,7	3,24
prompt=prosty	0,1	float16	0,8143	0,6674	0,7668	399,8	3,24
temp=0.0	0,0	float16	0,8127	0,6626	0,7636	435,5	3,24
temp=0.1	0,1	float16	0,8111	0,6576	0,7605	410,8	3,24
quant=float16	0,1	float16	0,8111	0,6575	0,7607	412,2	3,25
quant=4bit	0,1	4bit	0,7866	0,6284	0,7365	798,9	1,31
parsowanie=JSON	0,1	float16	0,5016	0,5050	0,5295	434,9	3,33

Główna obserwacja: Optymalną konfiguracją dla badanego modelu okazało się zastosowanie szczegółowego promptu w języku polskim przy niskiej temperaturze (0.1) w precyzji float16, co pozwoliło na osiągnięcie najwyższej celności na poziomie 85,8% oraz miary F1 macro = 80,0%.

8.8. Interpretacja wyników oraz wnioski

1. **Inżynieria promptów jako kluczowy czynnik sukcesu:** Zmiana treści instrukcji przyniosła najbardziej spektakularny zysk jakościowy. Przełamanie barier językowych w opisie klas pozwoliło modelowi Qwen2.5 na pełne wykorzystanie wiedzy semantycznej o języku polskim.

2. **Umiarkowany i probabilistyczny wpływ temperatury:** Zmiana temperatury w zakresie 0.0–0.1 nie wpływa na model z uwagi na deterministyczne dekodowanie zachłanne. Z kolei podbicie parametru do $T = 0,7$ rozszerzyło przestrzeń poszukiwań tokenów, co poskutkowało nieznacznym wzrostem miary F1 macro. Eksperyment ten dowodzi, że wyższa temperatura może pomóc w klasyfikacji klas rzadkich (np. neutral), kosztem utraty powtarzalności wyników.

3. **Składniowa destabilizacja przez format JSON:** Narzucenie strukturyzacji wyjścia za pomocą JsonOutputParser doprowadziło do załamania wydajności modelu do poziomu losowego (50,16%). Wynik ten jednoznacznie definiuje ograniczenia architektur o rozmiarze 1.5B parametrów, dbanie o techniczną poprawność kodu JSON odbywa się kosztem zdolności logicznego wnioskowania.

4. **Wydajnościowy kompromis kwantyzacji:** Przejście na format 4-bitowy NF4 to klasyczny, inżynierski kompromis. Zmniejszenie zużycia pamięci VRAM o blisko 60% (do poziomu zaledwie 1,31 GB) okopane jest spadkiem celności o ok. 2,5 p.p. oraz drastycznym, dwukrotnym wydłużeniem czasu pracy potoku (798,9 s). Kwantyzacja jest zatem doskonałym wyborem przy restrykcyjnych ograniczeniach

sprzętowych, ale nie sprawdza się przy optymalizacji systemów pod kątem czystej jakości i szybkości.

9. Porównanie encoder vs decoder

Kryterium	Encoder (HerBERT)	Decoder (Qwen LLM, zad. 4)
Accuracy	0,7590	0,8697
F1 macro	0,6263	0,8106
F1 weighted	0,7294	0,8602
F1 minus	0,91	0,96
F1 neutral	0,20	0,62
F1 plus	0,78	0,85
Zbiór testowy	614 próbek	614 próbek
Szybkość	Szybki (batch 16)	Wolny (≈ 1 tekst/sek.)
Konfiguracja	Pipeline, zero config	Prompt + parsowanie
GPU RAM	$\approx 0,5$ GB	≈ 3 GB (1,5B params)

Kluczowe obserwacje i dyskusja

- **Dominacja jakościowa modeli LLM:** Model dekodujący Qwen2.5 po wdrożeniu poprawnej warstwy parsowania tekstu bezapelacyjnie przewyższa model enkoderowy HerBERT we wszystkich globalnych metrykach klasyfikacyjnych.
- **Przełom w detekcji klasy neutralnej:** Największą słabością modelu enkoderowego była klasyfikacja tekstów obiektywnych i opisowych. HerBERT zanotował krytycznie niski wynik $F1 = 0,20$, permanentnie myląc klasę neutral z klasą plus (aż 95 pomyłek na 117 próbek). Model Qwen2.5 dzięki znacznie większej pojemności semantycznej poradził sobie z tym wyzwaniem bez porównania lepiej ($F1 = 0,62$). Choć on również przejawia tendencję do nadinterpretacji uprzejmego tonu jako sentymentu dodatniego (47 pomyłek), to poprawnie zidentyfikował połowę próbek neutralnych (58/117).
- **Efektywność ekonomiczna i wdrożeniowa enkoderów:** Choć architektury decoder-only wygrywają pod kątem czystej jakości predykcji, HerBERT pozostaje bezkonkurencyjny w kategoriach czysto inżynierskich. Wymaga blisko 7-krotnie mniej pamięci GPU ($\approx 0,5$ GB vs $\approx 3,25$ GB), nie potrzebuje budowania skomplikowanych potoków instrukcji (promptów) i pozwala na natywne przetwarzanie potokowe (batch inference), co czyni go rozwiązaniem wielokrotnie tańszym i szybszym w środowisku produkcyjnym.
- **Krytyczna rola potoku przetwarzania (Parsowanie i Domena):**
Eksperymenty jednoznacznie dowiodły, że sukces modeli LLM w zadaniach deterministycznych w 100% zależy od czyszczenia danych wyjściowych

W przypadku enkoderów kluczowa jest z kolei zgodność domenowa danych treningowych.

Wnioski końcowe

Wybór między enkoderem a dekodery w analizie sentymentu zależy bezpośrednio od priorytetów projektowych:

- Jeśli celem nadrzędnym jest maksymalna dokładność, poprawna interpretacja tekstów neutralnych oraz elastyczność bez konieczności ponownego trenowania sieci, najlepszym wyborem są małe modele LLM (Decoder) sterowane precyzyjnym promptem.
- Jeśli system ma działać w trybie czasu rzeczywistego, przy ograniczonych zasobach budżetowych/sprzętowych, mniejsze modele dedykowane BERT (Encoder) dostrojone do właściwej domeny językowej wciąż stanowią najbardziej optymalne i stabilne rozwiązanie inżynierskie.

10. Podsumowanie i wnioski

1. **Środowiska produkcyjne o niskiej latencji:** W systemach komercyjnych, gdzie kluczowy jest krótki czas odpowiedzi, wysoka przepustowość (przetwarzanie paczkami) oraz minimalne koszty infrastrukturalne, optymalnym wyborem pozostaje **architektura typu encoder-only (HerBERT)**. Zapewnia ona stabilne $\approx 76\%$ celności przy minimalnym narzucie na alokację pamięci GPU ($\approx 0,5$ GB) i zerowej podatności na błędy parsowania tekstu.

2. **Systemy nastawione na maksymalną jakość predykcji:** W systemach analitycznych typu offline, gdzie priorytetem jest precyzja (zwłaszcza w wyłapywaniu niuansów i tekstów neutralnych), a czas inferencji rzędu kilkunastu minut na kilkaset próbek jest akceptowalny, bezkonkurencyjne są modele LLM (Decoder).

3. **Optymalizacja sprzętowa na brzegu sieci:** Kwantyzacja do formatu 4-bitowego (NF4) jest wysoce rekomendowaną praktyką inżynierską w sytuacjach restrykcyjnych ograniczeń budżetowych. Umożliwia ona redukcję progu wejścia pamięci VRAM z 3,3 GB do zaledwie 1,3 GB, co pozwala na lokalne uruchamianie klasyfikatorów LLM nawet na konsumenckich lub starszych układach GPU, przy w pełni akceptowalnym koszcie jakościowym ($\approx -2,5$ p.p. Accuracy).

4. **Złota zasada integracji z modelami generatywnymi:** Podczas implementacji potoków klasyfikacyjnych opartych na mniejszych modelach LLM (klasy 1.5B), należy bezwzględnie unikać parserów obiektowych (np. JsonOutputParser), które drastycznie destabilizują proces generowania. Najbardziej niezawodnym i wydajnym podejściem jest zmuszenie modelu do zwrotu pojedynczego tokenu tekstowego i jego późniejsza izolacja za pomocą prostych wyrażeń regularnych.